

STATEMENTS, EXPRESSIONS, AND OPERATORS

Outline

- Statements
 - ✓ Statements and white spaces
 - ✓ Null statements
 - ✓ Compound statements
- Expressions
 - ✓ Simple Expressions
 - ✓ Complex Expressions
- Operators
 - ✓ Assignment operator
 - ✓ Mathematical operators
 - ✓ Operator precedence and parentheses
 - ✓ Order of subexpression evaluation
 - ✓ Relational operators
- The if statement
 - ✓ The else clause
- Evaluating relational expressions
- Logical operators
- True/False values
 - ✓ The Precedence of operators
 - ✓ Compound assignment operator
 - ✓ Conditional operator
 - ✓ Comma operator

Statements

- ✓ C programs consist of statements, and most statements are composed of expressions and operators.
- ✓ You need to understand these three topics in order to be able to write C programs.
- ✓ **Dfn:** *A statement is a complete direction instructing the computer to carry out some task.*
- ✓ *C, statements are usually* written one per line, although some statements span multiple lines.
- ✓ C statements always end with a semicolon (except for preprocessor directives such as `#define` and `#include`
- ✓ You've already been introduced to some of C's statement types in the previous lectures.
- ✓ For example: `x = 2 + 3;` is an assignment statement.
- ✓ It instructs the computer to add 2 and 3 and to assign the result to the variable x.

Statements and white spaces

- ✓ The term *white space* refers to spaces, tabs, and blank lines in your source code.
- ✓ *The C compiler isn't sensitive* to white space.
- ✓ When the compiler reads a statement in your source code, it looks for the characters in the statement and for the terminating semicolon, but it ignores white space.
- ✓ The statement **`x=2+3;`** is equivalent to this statement: **`x = 2 + 3;`**
- ✓ This gives you a great deal of flexibility in formatting your source code.
- ✓ Statements should be entered one per line with a standardized scheme for spacing around variables and operators.
- ✓ The point is to keep your source code readable.

- ✓ However, the rule that C doesn't care about white space has one exception: Within literal string constants, tabs and spaces aren't ignored; they are considered part of the string.
- ✓ *A string is a series/group of characters.*
- ✓ *Literal string* constants are strings that are enclosed within quotes and interpreted literally by the compiler, space for space.
- ✓ `printf("Hello, world!");` is not equivalent to `printf("Hello,      world!");`

Null Statements

- ✓ If you place a semicolon by itself on a line, you create what we call a *null statement*.
- ✓ *A statement that doesn't perform any action.*
- ✓ This is perfectly legal in C.
- ✓ In the coming lectures, you will learn how the null statements can be useful.

Compound Statements

✓ A compound statement, also called a ***block***, is a group of two or more C statements enclosed in braces.

✓ Here's an example of a block:

```
{  
    printf("C Programming, ");  
    printf("is good!");  
}
```

✓ It's a good idea to place braces on their own lines, making the beginning and end of blocks clearly visible.

✓ Placing braces on their own lines also makes it easier to see whether you've left one out.

✓ Note:

- i. Stay consistent with how you use white space in statements.
- ii. Put block braces on their own lines. This makes the code easier to read.
- iii. Line up block braces so that it's easy to find the beginning and end of a block.
- iv. Do not spread a single statement across multiple lines if there's no need to do so. Limit statements to one line if possible.

Expressions

- An *expression* is anything that evaluates to a numeric value.
- *C expressions* come in all levels of complexity.
 - i. Simple expressions
 - ii. Complex expressions

Simple Expressions

- ✓ The simplest C expression consists of a single item like a simple variable, literal constant, or symbolic constant.

Complex Expressions

- ✓ *Complex expressions consist of simpler expressions connected by operators.*
- ✓ *For example: $2 + 8$ is an expression consisting of the subexpressions 2 and 8 and the addition operator +.*
- ✓ *When an expression contains multiple operators, the evaluation of the expression depends on **operator precedence**.*
- ✓ *C expressions get even more interesting.*
- ✓ *Look at the following assignment statement: $x = a + 10$;*
- ✓ *This statement evaluates the expression $a + 10$ and assigns the result to x.*
- ✓ *The entire statement $x = a + 10$ is itself an expression that evaluates to the value of the variable on the left side of the equal sign.*
- ✓ *You can write statements such as **$y = x = a + 10$** ; which assigns the value of the expression $a + 10$ to both variables, x and y:*
- ✓ *You can also write statements such as this: **$x = 6 + (y = 4 + 5)$** ;*
- ✓ *The result of this statement is that y has the value 9 and x has the value 15.*
- ✓ *Note the parentheses, which are required in order for the statement to compile.*

Operators

- ✓ An **operator** is a symbol that instructs *C* to perform some operation, or action, on one or more operands.
- ✓ An **operand** is something that an operator acts on.
- ✓ In *C*, all operands are expressions.
- ✓ *C* operators fall into several categories:
 - i. The assignment operator
 - ii. Mathematical operators
 - iii. Relational operators
 - iv. Logical operators

Assignment Operator

- ✓ The *assignment operator is the equal sign (=)*.
- ✓ *Its use in programming is somewhat different from its use in regular math.*
- ✓ If you write `x = y;` in a C program, it doesn't mean "x is equal to y." Instead, it means "assign the value of y to x."
- ✓ In a C assignment statement, the right side **can be** any expression, and the left side **must be** a variable name.
- ✓ Thus, the form is as follows: ***variable = expression;***
- ✓ When executed, *expression is evaluated, and the resulting value is assigned to variable.*

Mathematical Operators

- ✓ C's mathematical operators perform mathematical operations such as addition and subtraction.
- ✓ The mathematical operators are categorized into two:
 - i. Unary mathematical operators (2)
 - ii. Binary mathematical operators (5)

Unary Mathematical Operators

- ✓ The *unary mathematical operators* are so named because they take a single operand.
- ✓ C has two unary mathematical operators

Operator	Symbol	Action	Examples
Increment	++	Increments the operand by one	++x, x++
Decrement	--	Decrements the operand by one	--x, x--

- ✓ The increment and decrement operators can be used only with variables, not with constants.
- ✓ The operation performed is to add one to or subtract one from the operand.
- ✓ The statements ++x; --y; are the equivalent of these statements:
 - ✓ $x = x + 1;$
 - ✓ $y = y - 1;$

- ✓ You should note that either unary operator can be placed before its operand (*prefix mode*) or after its operand (*postfix mode*).
- ✓ *These two modes are not equivalent.*
- ✓ *They differ in terms of when the increment or decrement is performed:*
- ✓ When used in prefix mode, the increment and decrement operators modify their operand before it's used.
- ✓ When used in postfix mode, the increment and decrement operators modify their operand after it's used.
- ✓ An example should make this clearer. Look at these two statements: `x = 10; y = x++;`
- ✓ After these statements are executed, x has the value 11, and y has the value 10.
- ✓ The value of x was assigned to y, and then x was incremented.
- ✓ In contrast, the following statements `x = 10; y = ++x;` result in both y and x having the value 11.
- ✓ x is incremented, and then its value is assigned to y.

Binary Mathematical Operators

- ✓ C's binary operators take two operands. The binary operators, which include the common mathematical operations
- ✓ *C has five binary mathematical operators*

Operator	Symbol	Action	Example
Addition	+	Adds two operands	$x + y$
Subtraction	-	Subtracts the second operand from the first operand	$x - y$
Multiplication	*	Multiplies two operands	$x * y$
Division	/	Divides the first operand by the second operand	x / y
Modulus	%	Gives the remainder when the first operand is divided by the second operand	$x \% y$

- ✓ The first four operators listed in the table should be familiar to you, and you should have little trouble using them.
- ✓ The fifth operator, modulus, might be new. Modulus returns the remainder when the first operand is divided by the second operand.
- ✓ For example, 11 modulus 4 equals 3 (that is, 4 goes into 11 two times with 3 left over).

Operator precedence

- ✓ In an expression that contains more than one operator, what is the order in which operations are performed?
- ✓ The order in which the operation is performed depends on the precedence of each of the operators in a given statement.
- ✓ The importance of this question is illustrated by the following assignment statement:
 $x = 4 + 5 * 3;$
- ✓ Performing the addition first results in the following, and x is assigned the value 27: $x = 9 * 3;$
- ✓ In contrast, if the multiplication is performed first, you have the following, and x is assigned the value 19: $x = 4 + 15;$
- ✓ Therefore some rules are needed about the order in which operations are performed.
- ✓ This order, called *operator precedence*, each operator has a specific precedence.
- ✓ When an expression is evaluated, operators with higher precedence are performed first.

Precedence of Mathematical Operators

✓ The precedence of the 7 mathematical operators is indicated in the table below

Operators	Relative Precedence
++ --	1
* / %	2
+ -	3

✓ Looking at the table above, you can see that in any C expression, operations are performed in the following order

- i. Unary increment and decrement
- ii. Multiplication, division, and modulus
- iii. Addition and subtraction

✓ If an expression that contains more than one operator with the same precedence level, the operators are performed in left-to-right order as they appear in the expression.

- ✓ What if the order of precedence doesn't evaluate your expression as needed?
- ✓ Using the previous example, what if you wanted to add 4 to 5 and then multiply the sum by 3?
- ✓ C uses parentheses to modify the evaluation order.
- ✓ A subexpression enclosed in parentheses is evaluated first, without regard to operator precedence.
- ✓ Thus, you could write $x = (4 + 5) * 3;$
- ✓ The expression $4 + 5$ inside parentheses is evaluated first, so the value assigned to x is 27.
- ✓ You can use multiple and nested parentheses in an expression.
- ✓ When parentheses are nested, evaluation proceeds from the innermost expression outward. Look at the following complex expression:
- ✓ Example: $x = 25 - (2 * (10 + (8 / 2)));$
- ✓ **Which value is assigned to x ?**